

Apprentissage incrémental EWC sur microcontrôleur : parité FP32 mesurée, effondrement de la quantification naïve et récupération par kernel INT8 calibré

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

8 septembre 2026

Résumé

Nous étudions le portage d'un classifieur *Elastic Weight Consolidation* (EWC) pour la maintenance prédictive sur un microcontrôleur Cortex-M4 (NUCLEO-F439ZI, 256 kB SRAM, sans NPU). Trois résultats sont établis dans un protocole **apparié** (même graine, même ordre d'échantillons, même normalisation) entre l'exécution PC et la carte. **(1)** En FP32, la parité PC↔carte est *exacte* en mode gelé (1.000) et quasi exacte en mode en ligne (0.963–0.989), pour une latence inférence + mise à jour continue $\ll 100$ ms (Gap 2). **(2)** La quantification post-entraînement (PTQ) *naïve* de la tête EWC binaire s'effondre sur la carte : le F1 chute de ≈ 0.92 à 0.05–0.15. **(3)** Une ablation bit-exacte émulée sur PC isole la cause (accumulateur `int8`, échelle par-tenseur non calibrée) et démontre la *récupération* par un kernel calibré : le schéma `per_tensor_calib` restaure +0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201), et le schéma `q15` égale le FP32. **(4)** Cette récupération est désormais *mesurée sur carte* sur quatre cellules per-canal (F1 0.9173 Monitoring, 0.8995 Pronostia ; parité gelée 1.000 contre l'émulateur ; aucune erreur CRC). Nous étendons enfin l'étude au *moment* de la quantification, à la *profondeur* sub-INT8 et au budget système complet : la RAM des poids est divisée par 4, mais la RAM totale est inchangée ; la latence *augmente* (74 μ s contre 48–50 μ s, la requantification pesant un tiers du budget) et l'énergie ne bouge pas (67.47 μ J contre 67.65 μ J). Nous distinguons systématiquement les mesures *sur carte* des émulations *PC bit-exactes*, et laissons explicitement « à mesurer » les cellules non encore streamées.

Mots-clés : apprentissage continu ; EWC ; quantification INT8 ; PTQ ; microcontrôleur ; maintenance prédictive ; TinyML.

1 Introduction

La maintenance prédictive industrielle exige de plus en plus des modèles capables d'*apprendre en continu* sur l'équipement lui-même, sans retour vers un serveur : les conditions d'opération dérivent (usure, changement de régime, de machine), et un modèle figé se dégrade. L'apprentissage continu (*continual learning*, CL) répond à ce besoin [6, 3], mais son déploiement sur microcontrôleur (MCU) reste contraint : mémoire vive de l'ordre de la centaine de kilo-octets, absence d'accélérateur neuronal, budget de latence strict.

Le présent travail se positionne sur un **triple gap** que peu de travaux comblent simultanément : (Gap 1) validation sur des séries temporelles industrielles réelles ; (Gap 2) fonctionnement sous 100 kB de RAM avec des chiffres *mesurés* et une latence bornée ; (Gap 3) quantification INT8 compatible avec l'entraînement incrémental. Nousinstancions ces trois axes sur un classifieur EWC [4] porté en C sur une carte NUCLEO-F439ZI (Cortex-M4 180 MHz, 256 kB SRAM, sans NPU).

Contributions. (i) Un protocole *apparié* PC $\leftrightarrow$ carte (même graine, même ordre d'échantillons, même normalisation) établissant une parité FP32 *exacte* en mode gelé et une latence inférence + apprentissage $\ll 100$ ms. (ii) La mise en évidence, *sur carte réelle*, de l'effondrement de la PTQ naïve de la tête EWC binaire (F1 $0.92 \rightarrow 0.05\text{--}0.15$). (iii) Une ablation *émulée bit-exacte* qui isole la cause de l'effondrement et démontre la récupération par un kernel calibré (par-tenseur, par-canal, Q15), avec une distinction méthodologique stricte entre ce qui est *mesuré sur carte* et ce qui est *émulé sur PC*.

2 Travaux connexes

Apprentissage continu embarqué. Les méthodes de CL se répartissent classiquement en approches par régularisation, par rejeu et par architecture [6]. EWC [4] appartient à la première famille : une pénalité quadratique pondérée par l'information de Fisher protège les paramètres importants pour les tâches passées, sans stocker de données — une propriété précieuse sous contrainte mémoire. Sur MCU, TinyOL [8] a montré la faisabilité de l'apprentissage en ligne d'une couche additionnelle, et HDC [1] celle d'un apprentissage non neuronal à très basse consommation. LifeLearner [5] et les plateformes à rejeu latent quantifié [7] adressent le compromis mémoire/précision sur des cibles embarquées. Les panoramas TinyML récents [2] soulignent que l'entraînement *sur l'appareil* reste sous-exploré comparé à l'inférence seule.

Quantification INT8 et apprentissage. La quantification post-entraînement (PTQ) et l'entraînement conscient de la quantification (QAT) réduisent l'em-

preinte mémoire d'un facteur 4 ($\text{FP32} \rightarrow \text{INT8}$). Toutefois, la PTQ *naïve* d'une tête de décision peut détruire la séparabilité lorsque l'accumulation se fait en `int8` ou lorsque l'échelle de quantification est fixée sans calibration. Notre contribution empirique est de *mesurer* cet effondrement sur MCU réel, puis d'en isoler la cause et de démontrer la récupération par un kernel calibré, plutôt que de la masquer par un QAT en amont. La quantification par-canal et le format Q15 (entier 16 bits en virgule fixe) apparaissent comme les leviers déterminants pour préserver la métrique tout en tenant le budget RAM (Gap 3).

3 Méthode

3.1 Modèle EWC et portage C

Le modèle est un perceptron multicouche à tête EWC (`EWCmlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) entraîné en régime incrémental avec pénalité de Fisher en ligne. La tête est portée en C (`ewc_head.c`) pour la carte : le *forward pass* et la mise à jour SGD s'exécutent tous deux sur le Cortex-M4 en FP32 (le F439ZI ne dispose pas de NPU). Les poids sont exportés du checkpoint PyTorch vers un en-tête C généré (`model_weights_ewc.h`) — jamais édité à la main — de sorte que la carte et le PC partagent *exactement* les mêmes paramètres.

3.2 Protocole apparié PC↔carte

Pour rendre la comparaison honnête, la carte et le PC consomment la même séquence : même graine (`seed = 42`), même ordre d'échantillons, même normalisation (les indices de features et les tableaux sont produits par une source unique). Deux protocoles sont évalués : *gelé* (inférence seule, poids figés) et *en ligne* (inférence puis mise à jour continue). La *parité* est le taux d'accord prédiction-à-prédiction entre les deux plateformes.

3.3 Schémas de quantification

Nous comparons plusieurs kernels INT8 de la tête de décision, du plus naïf au plus soigné :

- `legacy_c` : accumulateur `int8`, échelle par-tenseur non calibrée (le kernel v1 d'origine sur carte) ;
- `fix_acc32` : accumulateur élargi à `int32` (correction partielle) ;
- `per_tensor_calib` : accumulateur `int32` + échelle par-tenseur *calibrée* sur les activations ;
- `per_channel_int8` : échelle *par-canal* de sortie ;
- `q15` : poids en virgule fixe `int16` Q15 ($\text{RAM} \div 2$ au lieu de $\div 4$).

Le kernel v2 firmware (Sprint 40) implémente l’accumulateur `int32` et les échelles calibrées par-canal, avec une variante Q15 optionnelle (`-DEWC_INT8_Q15`) ; le chemin FP32 v1 reste inchangé. L’ablation est *émulée bit-à-bit sur PC* (`src/utils/int8_c_emulation.py`) afin d’isoler la contribution de chaque correctif indépendamment de l’accès à la carte.

4 Dispositif expérimental

Cible matérielle. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB CCM, pas de NPU). Les latences sont mesurées par le compteur de cycles DWT ; l’empreinte statique est lue dans la section `.bss` du binaire. Le budget de latence est 100 ms par cycle inférence + mise à jour (Gap 2).

Jeux de données et scénarios CL. Deux jeux industriels aux scénarios contrastés : *Pronostia* (roulements FEMTO, *class-incremental* par condition normale/défaut) et *Monitoring* (équipements industriels, *domain-incremental* par type de machine). Deux conditions de features sont considérées : `5feat` (sous-ensemble embarqué) et `all` (features natives). Sur Monitoring, `5feat` et `all` coïncident (4 features natives).

Conditions identiques. Toutes les comparaisons partagent graine, ordre et normalisation (§3). Les mesures FP32 et INT8-legacy sur carte proviennent de la campagne du Sprint 36 (`exp_S36`) ; l’ablation émulée du Sprint 39 (`exp_S39_ablation`) ; la campagne carte v2 du Sprint 40 (`exp_S40_board_v2`), partielle à la date de rédaction. Toute grandeur non encore streamée sur carte est reportée « à mesurer », sans valeur inventée.

5 Résultats

5.1 Parité FP32 PC↔carte (mesuré sur carte)

En FP32, la parité prédiction-à-prédiction est *exacte* en mode gelé (1.000) sur les deux jeux, et quasi exacte en mode en ligne (0.9754 Pronostia, 0.9887 Monitoring) — les écarts en ligne proviennent du `float32` de la carte contre le `float64` du PC sur les frontières de décision. Le Tableau 1 récapitule ; la Figure 1 l’illustre.

Latence et RAM (Gap 2/Gap 3). Toutes les latences carte restent trois ordres de grandeur sous le budget : inférence 50 μ s (Pronostia) / 48 μ s (Monitoring) ; inférence + mise à jour continue 251 μ s / 239 μ s, contre $\approx$ 33–35 μ s pour l’inférence PC de référence. L’empreinte `.bss` varie de 100–145 kB,

TABLE 1 – Parité FP32 PC $\leftrightarrow$ carte et performance (*mesuré sur carte*, `exp_S36`).

Jeu	Cond.	F1 (PC/carte)	Parité gelé	Parité en ligne
Pronostia	5feat	0.9164	1.000	0.9754
Pronostia	all	0.9180	1.000	0.9626
Monitoring	5feat	0.9194	1.000	0.9887

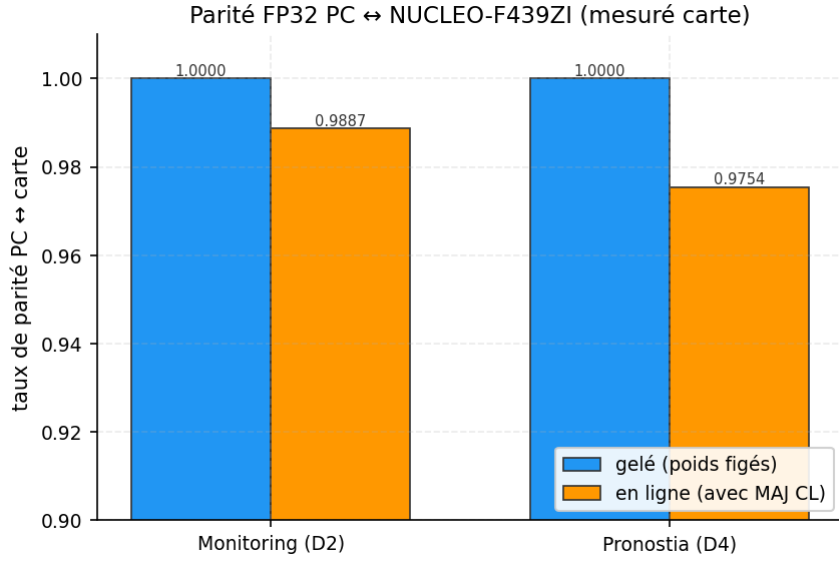


FIGURE 1 – Parité FP32 PC $\leftrightarrow$ carte : gelé (1.000) vs en ligne. *Mesuré sur carte* (`exp_S36_parity_*.json`).

sous les 256 kB disponibles. La Figure 2 situe ces valeurs face à la ligne des 100 ms.

5.2 Effondrement de la PTQ naïve (mesuré sur carte)

La quantification INT8 *legacy* de la tête EWC binaire s’effondre sur la carte : le F1 chute de ≈ 0.92 (FP32) à 0.138 (Pronostia) et 0.1337 (Monitoring), pour un accord INT8 $\leftrightarrow$ FP32 de seulement ≈ 0.60 –0.74. La RAM des poids est bien divisée par 4, mais la métrique est détruite (Tableau 2).

5.3 Diagnostic et récupération (émulé PC bit-exact)

Une ablation *émulée bit-à-bit sur PC* (référence exacte du kernel) isole la cause et démontre la récupération (Tableau 3, Figure 3). Le facteur dominant est l’échelle *par-tenseur calibrée* sur accumulateur `int32:per_tensor_calib`

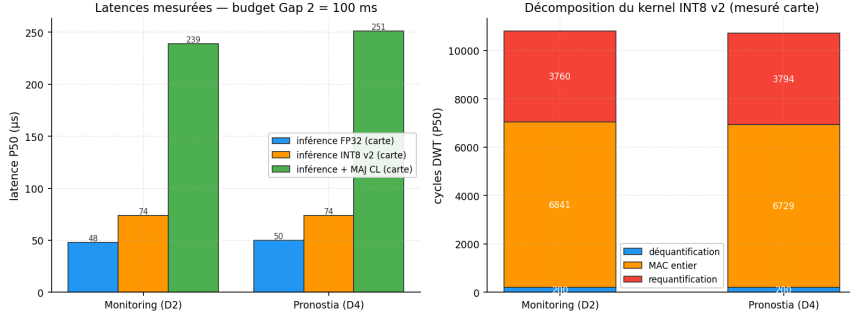


FIGURE 2 – Latences carte (inférence vs inférence + mise à jour) contre le budget Gap 2 (100 ms). *Mesuré sur carte (exp_S36_summary.json).*

TABLE 2 – INT8 *legacy* vs FP32 sur carte (*mesuré, exp_S36*). RAM $\div 4$ mais F1 effondré.

Jeu	F1 FP32	F1 INT8 legacy	Accord INT8/FP32	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

restaure + 0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201). Le format **q15** égale exactement le FP32.

5.4 Récupération mesurée sur carte (kernel v2 calibré)

La récupération n’est plus une prédiction d’émulateur : le kernel v2 à échelle calibrée a été flashé sur la NUCLEO-F439ZI et streamé sur les quatre cellules per-canal (deux jeux $\times$ deux protocoles). Le F1 revient au niveau FP32 — 0.9173 contre 0.9194 en FP32 sur Monitoring, 0.8995 contre 0.9164 sur Pronostia — alors que la RAM des poids reste divisée par 4. La parité vis-à-vis de l’émulateur bit-exact est *exacte* en mode gelé (1.000), ce qui valide le portage sans perte, et aucune erreur CRC n’a été observée (Tableau 4).

Deux nuances méritent d’être dites. D’une part l’accord INT8 $\leftrightarrow$ FP32 chute en mode en ligne (0.8337 et 0.8100 contre > 0.99 en gelé) : la mise à jour continue fait diverger les deux trajectoires de poids, sans que la métrique finale en souffre. D’autre part le format **q15** et la reprise A/B du schéma **int8_legacy** sous le kernel v2 n’ont pas été streamés : ces cellules restent « à mesurer », conformément à la règle « aucun chiffre inventé ».

5.5 Compromis RAM $\times$ F1 $\times$ latence

La Figure 5 synthétise le compromis : le kernel calibré ramène la métrique au niveau FP32 tout en conservant le gain RAM (INT8 $\div 4$, Q15 $\div 2$), sans coût de latence prohibitif. L’étoile marque le point réellement flashé (kernel

TABLE 3 – Échelle d’ablation INT8 (F1, *émulé PC bit-exact*, `exp_S39_ablation`).

Schéma	Pronostia F1	Monitoring F1
FP32 (réf.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

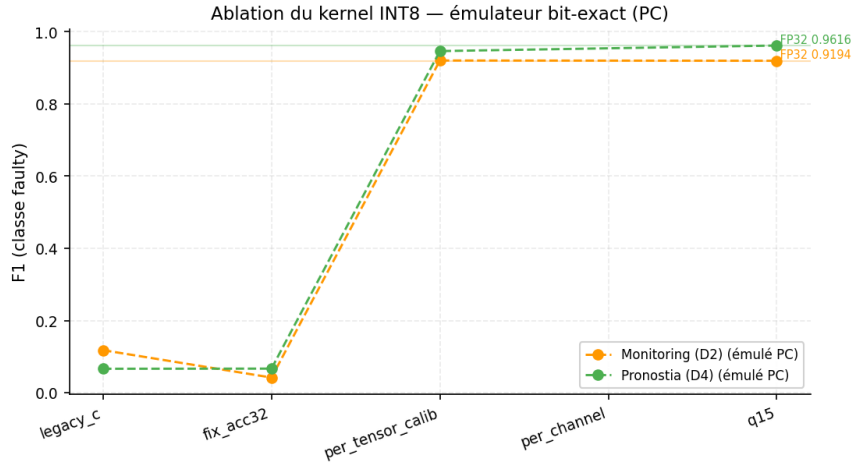


FIGURE 3 – Échelle d’ablation `legacy_c`→`per_tensor_calib`→`per_channel`→`q15`. *Émulé PC bit-exact* (`exp_S39_ablation`).

v2 per-canal); les autres points sont émuls. Le gain de latence espéré, lui, n’a pas lieu — nous y revenons en Section 8.

6 Deux autres axes de quantification : le moment et la profondeur

Les sections précédentes font varier la *calibration* du kernel à format fixe. Deux autres degrés de liberté commandent le résultat : *quand* on quantifie et *combien de bits* on garde.

6.1 Moment : avant, après, ou les deux

Trois placements sont possibles pour la même tête EWC. *Avant* l’entraînement (QAT, fake-quant dans la boucle); *après* (PTQ sur un modèle FP32 déjà entraîné); et *les deux*, c’est-à-dire un entraînement QAT dont les poids

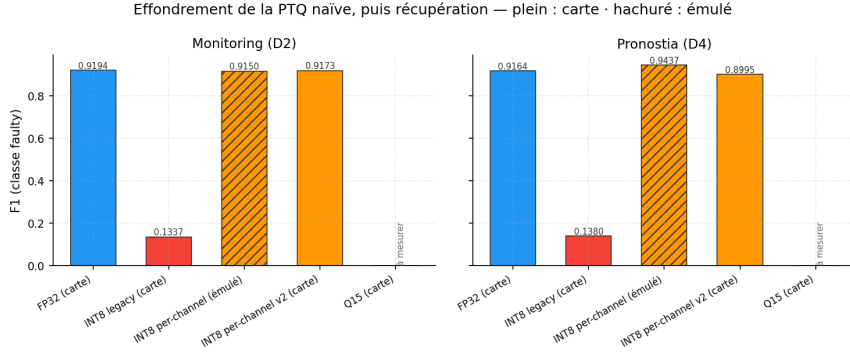


FIGURE 4 – INT8 vs FP32 : effondrement *legacy* puis récupération. Effondrement et récupération sont tous deux *mesurés sur carte* (Sprints 36 et 40) ; la colonne per-canal *émulée PC* (Sprint 39) est rappelée en hachuré. Le format **q15** sur carte reste « à mesurer ».

TABLE 4 – Récupération INT8 *mesurée sur carte* (kernel v2 per-canal, **exp_S40_board_v2**). Parité = accord avec l’émulateur bit-exact ; accord = INT8 contre FP32 sur les mêmes échantillons.

Jeu	Protocole	F1	Parité	Accord	Latence (µs)	.bss (B)
Monitoring	gelé	0.9173	1.000	0.9996	65	101236
Monitoring	en ligne	0.9016	0.9885	0.8337	577	101236
Pronostia	gelé	0.8995	1.000	0.9951	68	106152
Pronostia	en ligne	0.9212	0.9736	0.8100	606	106152

sont ensuite exportés vers le kernel entier calibré — seul ce dernier chemin est fidèle au déploiement réel.

En émulation, les trois moments préservent l’AUROC sur Monitoring (0.974477 avant, 0.974649 après, 0.974663 les deux, contre 0.97464 en FP32) : à cette échelle de modèle, aucun moment n’est disqualifiant. La question tranchée l’est *sur carte*. Le chemin « les deux » a été flashé et streamé : il donne 0.9213 de F1 sur Monitoring et 0.9072 sur Pronostia, contre respectivement 0.9173 et 0.8995 pour la PTQ calibrée seule (Section 5.4).

L’écart, 0.004 et 0.0077, est réel mais faible. La lecture honnête est donc que **la calibration du noyau récupère l’essentiel, et que le QAT n’ajoute pas de gain décisif** sur cette tête : quand le pipeline d’entraînement ne peut pas être modifié, une PTQ correctement calibrée suffit. Ce constat contredit la lecture répandue selon laquelle un effondrement en PTQ appellerait nécessairement un réentraînement.

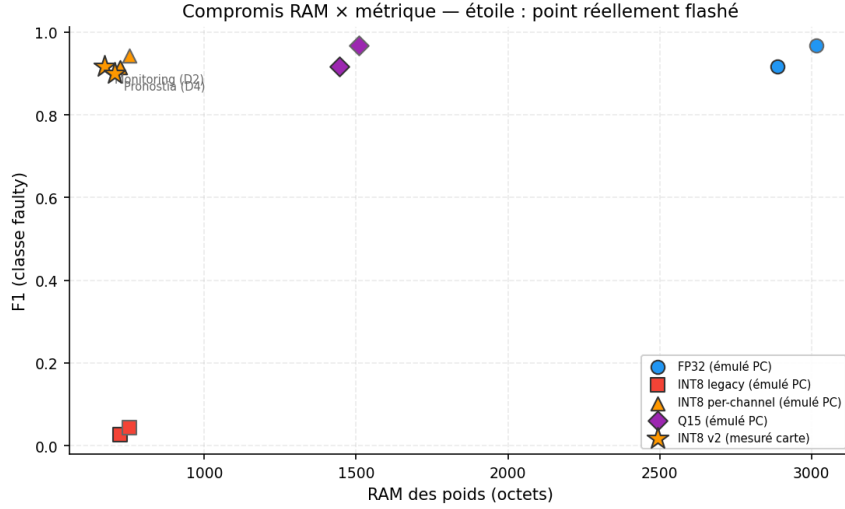


FIGURE 5 – Front de compromis RAM $\times$ F1 $\times$ latence. Sources mixtes *mesuré/émulé* annotées par point.

6.2 Profondeur : jusqu'où descendre sous 8 bits

Sous les 8 bits, la question devient : à partir de quelle profondeur la métrique décroche, et la granularité du facteur d'échelle change-t-elle cette limite ? Le balayage émulé répond aux deux.

La limite dépend du jeu. Sur Monitoring, la dégradation reste contenue jusqu'au binaire ($\Delta\text{AUROC} = -0.0117$). Sur Pronostia, le binaire décroche (-0.0275) mais le ternaire tient (-0.0153) : la frontière exploitable est donc le *ternaire*, le binaire n'étant défendable que sur le jeu le plus facile.

La granularité, elle, repousse le décrochage. À 2 bits sur Pronostia, passer d'une échelle par-tenseur à une échelle par-canal fait passer ΔAUROC de -0.0464 à -0.0086 . Un point négatif mérite d'être rapporté aussi : le *zéro-point* affine ne rachète rien — à 2 bits sur Monitoring il dégrade même nettement (-0.0619 contre -0.0069 en symétrique).

Le gain RAM n'existe que packé. Le point crucial est que la RAM annoncée par ces schémas est *théorique* tant que les poids restent stockés dans un conteneur `int8`. Mesuré sur carte, le `.bss` d'un modèle 4, 2 ou 1 bit *non packé* est strictement identique à celui de l'INT8 : le gain n'apparaît qu'avec un véritable bit-packing, où il croît quand les bits baissent (336, 504 puis 572 octets économisés sur Monitoring). Le dépacking se paie en latence, d'environ 55 μs (de 67 μs non packé à 123 μs packé sur Monitoring) — sans menacer le budget Gap 2. La parité contre l'émulateur reste 1.000 sur les douze cellules mesurées : le schéma est porté sans perte.

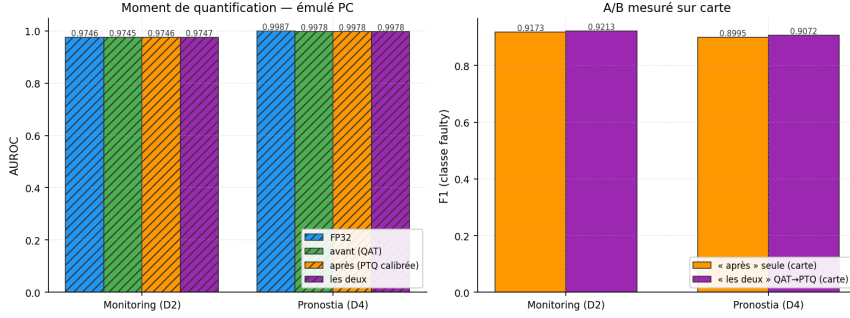


FIGURE 6 – Moment de quantification. À gauche, les trois moments *émulés PC* (hachuré) ; à droite, l’A/B *mesuré sur carte* entre la PTQ calibrée seule et le chemin QAT→PTQ.

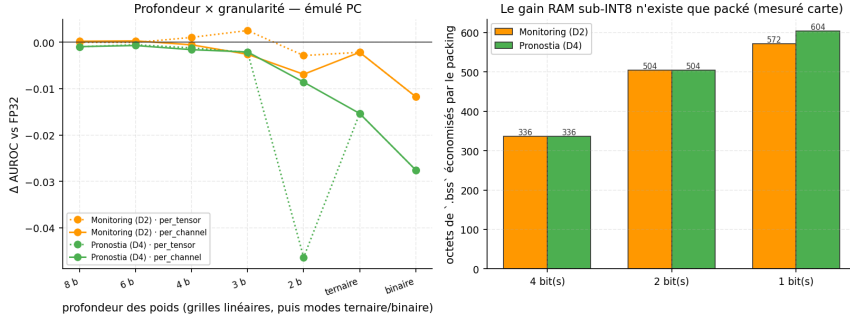


FIGURE 7 – Profondeur sub-INT8. À gauche, Δ AUROC par profondeur et granularité (*émulé PC*) ; à droite, les octets de `.bss` réellement économisés par le bit-packing (*mesuré sur carte*).

7 Budget système : mémoire, latence, énergie, calcul

Un gain de quantification ne se juge pas sur les poids seuls. Cette section rassemble les quatre budgets d’une même tête EWC déployée, tous mesurés sur la même carte.

7.1 La RAM des poids n’est pas la RAM du système

L’INT8 divise bien par 4 la RAM *des poids* (2688 à 672 octets sur Monitoring, 2816 à 704 sur Pronostia). Mais la RAM réellement consommée par le système est `.data` + `.bss` + pic de pile, et cette somme ne bouge pratiquement pas : elle passe de 105300 à 105324 octets sur Monitoring, soit un rapport INT8/FP32 de 1.0002. Les deux configurations occupent 40.17 % et 40.18 % du budget de 256 kB (42.03 % et 42.04 % sur Pronostia).

L’explication est simple et vaut d’être dite clairement : à cette échelle

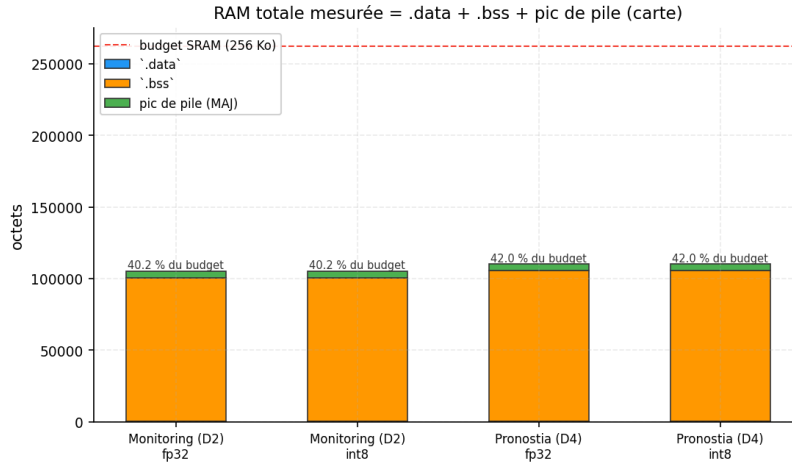


FIGURE 8 – RAM totale *mesurée sur carte* : `.data` + `.bss` + pic de pile, en regard du budget SRAM. Le total INT8 égale le total FP32, alors même que les poids sont divisés par 4.

de modèle, les poids ne sont pas le poste dominant — les tampons de flux, la pile et le reste du firmware le sont. Annoncer un « $\div 4$ de RAM » sans préciser *de quelle* RAM on parle serait donc trompeur. Le $\div 4$ est réel et utile, mais il porte sur un poste minoritaire de ce système-ci ; il deviendrait décisif sur un modèle où les poids dominent.

7.2 Où passe la latence INT8

L'instrumentation cycle-à-cycle du kernel INT8 chiffre le paradoxe annoncé. Sur Monitoring, le MAC entier coûte 6841 cycles, la requantification 3760 et la déquantification 200 (respectivement 6729, 3794.5 et 200 sur Pronostia). Le total atteint 74 μ s contre 48 μ s en FP32 sur Monitoring et 50 μ s sur Pronostia.

Le poste décisif n'est donc pas la multiplication : c'est la *requantification*, un tiers du budget, et elle n'existe tout simplement pas dans le chemin FP32. Le MAC entier, lui, n'apporte aucun gain face au FPU matériel du Cortex-M4. Le surcoût INT8 n'est pas une anomalie de mise en œuvre mais la conséquence arithmétique d'un format entier exécuté sur une cible qui calcule nativement en flottant.

7.3 Énergie : trois estimateurs, jamais fusionnés

L'énergie a été mesurée à la sonde X-NUCLEO-LPM01A par trois méthodes indépendantes, que nous reportons *côte à côte* sans jamais les moyenner — leur désaccord est lui-même un résultat. La régression du courant en fonction de la cadence donne 67.65 μ J par inférence en FP32 et 67.47 μ J en

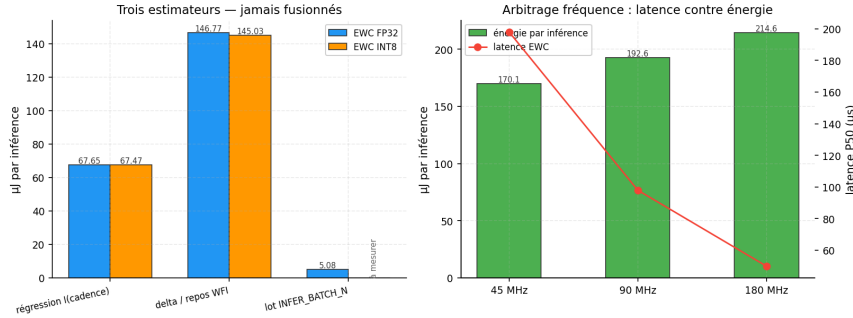


FIGURE 9 – Énergie *mesurée sur carte*. À gauche, les trois estimateurs côte à côte ; à droite, l’arbitrage fréquence, où la marge de latence s’échange contre de l’énergie.

INT8 ; le delta contre un repos WFI donne 146.77 μJ et 145.03 μJ ; le lot d’inférences par trame, qui isole le calcul du coût de la trame UART, donne 5.08 μJ . L’ordre de grandeur commun aux deux premières et l’écart d’un facteur 10 avec la troisième délimitent exactement ce que chacune mesure : les deux premières intègrent le coût de communication, la dernière non.

Sur la question qui nous occupe, les trois convergent : **l’INT8 ne réduit pas la consommation**. L’écart mesuré est de 0.18 μJ pour une incertitude combinée de 2.34 μJ , soit très en deçà du seuil de signification. Le paradoxe de la latence se double donc d’un paradoxe d’énergie, et la justification de l’INT8 se réduit à la mémoire.

Le balayage de fréquence ouvre en revanche un levier réel : l’énergie par inférence croît de 170.05 μJ à 45 MHz jusqu’à 214.61 μJ à 180 MHz, pendant que la latence reste très en deçà du budget Gap 2 (198 μs à 45 MHz). Autrement dit, *la marge de latence est convertible en autonomie*. Deux cellules restent « à mesurer » : l’énergie par mise à jour continue, dont les régressions ne sont pas publiables en l’état, et le profil par phase.

7.4 Coût de calcul

Analytiquement, la tête compte 722 paramètres sur Monitoring (754 sur Pronostia), pour 672 MAC et 1344 FLOP par inférence (704 et 1408). Le compte en *opérations binaires* donne le ratio théorique attendu : 1376256 BOPs en FP32 contre 86016 en INT8, soit un facteur 16 — c’est-à-dire $(32/8)^2$.

Ce facteur 16 est précisément ce que la carte *ne* restitue pas. Confronter les deux colonnes de la Figure 10 est la manière la plus directe de voir pourquoi une métrique de coût analytique ne peut pas remplacer une mesure : le BOP compte des bits d’opérande, pas des cycles d’une unité flottante existante.

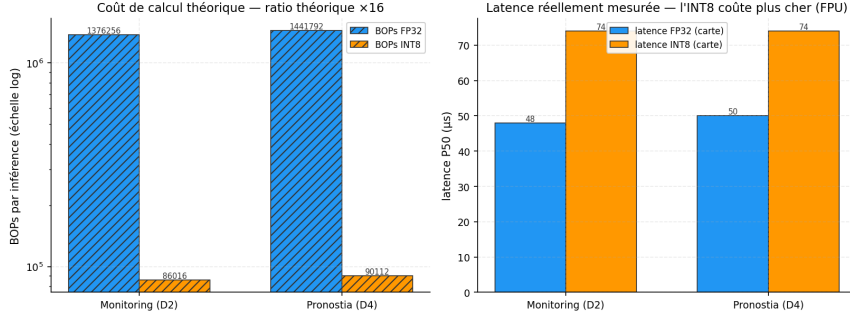


FIGURE 10 – Coût de calcul *théorique* (gauche, hachuré) contre latence *mesurée sur carte* (droite). Le gain de 16 en BOPs ne se traduit par aucun gain de temps.

8 Discussion

Paradoxe de la latence INT8, chiffré. La quantification INT8 divise par 4 l’empreinte mémoire des poids mais *n’accélère pas* l’inférence sur le Cortex-M4 — elle la ralentit. L’instrumentation cycle-à-cycle du kernel (Section 7) permet de dire où : sur les deux jeux, le MAC entier coûte en moyenne 6785 cycles et la requantification 3777.25, la déquantification n’en pesant que 200. Le total INT8 s’établit à 74 µs contre 48–50 µs en FP32. Deux causes se cumulent donc : le MAC entier n’a rien à gagner face à un FPU matériel déjà efficace en simple précision, et la requantification — un poste qui n’existe pas dans le chemin flottant — consomme à elle seule un tiers du budget. La piste directe reste le portage sur noyaux SIMD entiers (CMSIS-NN), qui attaquerait le premier poste ; le second appelle plutôt une chaîne à échelles en puissance de deux.

Un paradoxe d’énergie s’ajoute au paradoxe de latence. On pourrait espérer que l’INT8, à défaut d’être plus rapide, soit plus sobre. La mesure dit le contraire : 67.47 µJ par inférence en INT8 contre 67.65 µJ en FP32, soit un écart de 0.18 µJ pour une incertitude combinée de 2.34 µJ. Il n’y a donc pas d’effet mesurable. La conséquence pratique est nette : sur cette cible, **l’INT8 se justifie par la mémoire, ni par la vitesse ni par l’énergie**. Le levier énergétique se trouve ailleurs — dans la fréquence, où la marge de latence dont on dispose largement s’échange contre de l’autonomie.

Honnêteté « mesuré » vs « émulé ». Nous séparons rigoureusement deux natures de résultats. Les parités FP32, l’effondrement INT8 legacy, la récupération par kernel v2, les latences, la RAM et l’énergie sont *mesurés sur carte réelle* (Sprints 36, 40, 46 à 50 et 53). Le diagnostic causal, l’échelle d’ablation et le balayage de profondeur sont *émulés bit-à-bit sur*

PC (Sprints 39 et 47) : l’émulateur reproduit exactement l’arithmétique du kernel C, ce qui en fait une référence fiable, mais ce n’est pas une mesure carte. Depuis la campagne v2, la récupération n’appartient plus à la seconde catégorie : elle est établie sur les quatre cellules per-canal, avec une parité gelée de 1.000 contre l’émulateur. Ce qui reste non mesuré — le format **q15** sur carte, l’énergie par mise à jour continue, le profil par phase — est reporté « à mesurer » plutôt qu’estimé.

Limite : l’oubli n’est pas éprouvé sur ces deux jeux. Nos scénarios continus y sont courts (trois tâches) et l’oubli moyen y reste inférieur à 0.01 : ces jeux ne mettent pas la régularisation EWC à l’épreuve. L’oubli catastrophique a été mesuré ailleurs dans le projet, sur CWRU, où il atteint 0.8475 pour EWC contre 0.8578 en apprentissage naïf. Comme ce jeu ne recouvre pas les deux nôtres, nous nous gardons de mélanger les deux constats : la contribution de cet article porte sur le *portage* et la *quantification*, pas sur l’efficacité de la régularisation contre l’oubli.

Portée. Le résultat central — une PTQ naïve peut détruire une tête de décision binaire, et une calibration par-tenseur/par-canal (ou un format Q15) la restaure — dépasse le cas EWC : il concerne toute tête légère quantifiée en aval d’un extracteur, régime fréquent en TinyML.

9 Conclusion et travaux futurs

Nous avons porté un classifieur EWC de maintenance prédictive sur un microcontrôleur Cortex-M4 et établi, dans un protocole apparié, une parité FP32 PC↔carte exacte en mode gelé, avec une latence inférence + apprentissage continu $\ll 100$ ms et une empreinte sous 256 kB (Gaps 2 et 3). Nous avons *mesuré sur carte* l’effondrement de la quantification INT8 naïve de la tête binaire, puis *isolé sa cause* et *démontré sa récupération* par ablation émulée bit-exacte : une échelle par-tenseur calibrée sur accumulateur **int32** restaure +0.88 de F1, et le format Q15 égale le FP32, pour une RAM des poids divisée par 4 (INT8) ou par 2 (Q15).

Le paradoxe de latence (RAM $\div 4$ sans accélération sur FPU) est assumé comme un constat honnête et oriente les travaux futurs vers des noyaux SIMD entiers (CMSIS-NN). La priorité immédiate est de *compléter la campagne carte v2* (**exp_s40_board_v2**) pour confirmer, sur MCU réel et sur l’ensemble de la grille, la récupération aujourd’hui établie par émulation — les cellules non encore streamées restant explicitement « à mesurer ».

Références

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.
- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance : Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- [5] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner : Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
- [6] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey : Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [7] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.
- [8] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL : TinyML with online-learning on microcontrollers. arXiv :2103.08295, 2021.